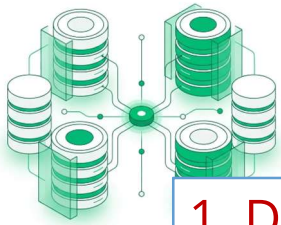


Bases de Datos Relacionales

Introducción. DISEÑO FÍSICO



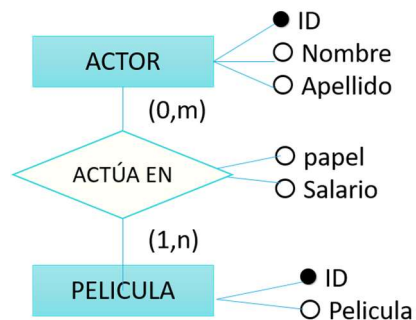
Fases del Diseño de una Base de Datos

1. Descripción de requisitos

2. Diseño conceptual

3. Diseño lógico

4. Diseño físico



ESQUEMA CONCEPTUAL

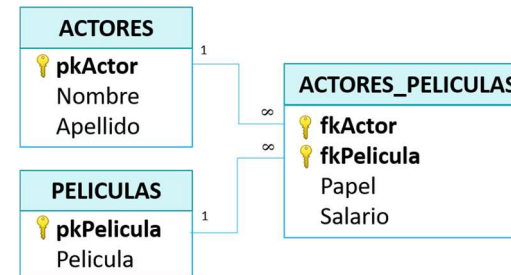
ACTORES (pkActor, Nombre, Apellido)

PELICULAS (pkPelícula, Película)

ACTORES-PELICULAS (fkActor, fkPelícula, Papel, Salario)

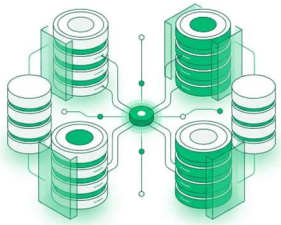
donde
{fkActor} referencia ACTORES
{fkPelícula} referencia PELICULAS

ESQUEMA LÓGICO



ESQUEMA FÍSICO



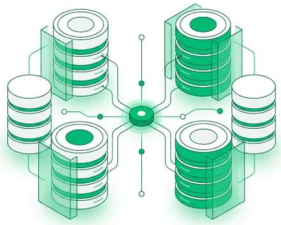


<https://www.postgresql.org/>

Diseño Físico de la Base de Datos



En esta fase,
se crean
las estructuras
**en el Motor de
Base de Datos**
que se va
a utilizar,
**desde el Gestor
seleccionado.**



Usaremos **SupaBase**
PostGres en la Nube

<https://supabase.com/>

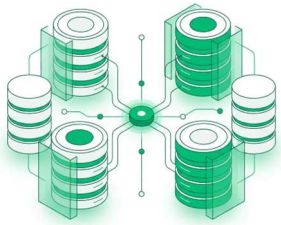


Build in a weekend Scale to millions

Start your project with a Postgres database. Add Authentication, Data APIs, Edge Functions, Realtime Data, Storage, and Vector embeddings.

Start your project

Request a demo

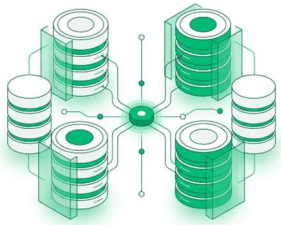


Crear un "Proyecto" en Supabase

Cuando se crea un proyecto en Supabase, la plataforma aprovisiona automáticamente una instancia física completa de PostgreSQL con su base de datos principal llamada **postgres**.

Para ello:

1. Entra al panel de Supabase (database.new).
2. Haz clic en **New Project**.
3. Dale un nombre al proyecto y una contraseña maestra para la Base de Datos.
4. Eliges la región (en nuestro caso: Spain - Madrid).
5. En unos 60 segundos, Supabase crea la infraestructura y entrega la base de datos lista para usar.



Diseño Físico de la Base de Datos

EMPLEADOS (dni, código, nombre, apellido, fecha_nacimiento, calle/av/via, numero, edf/casa, escalera, planta, puerta, cp, **ciudad**, sueldo, código_dpto, fecha_inicio, num-sso)

donde

{código_dpto} referencia departamento

TELEFONOS_EMPLEADOS (dni + telefono)

donde

{dni} referencia EMPLEADO

PROYECTOS (id proyecto, proyecto, descripción)

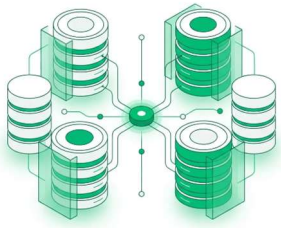
PROYECTOS_EMPLEADO (id proyecto + dni)

donde

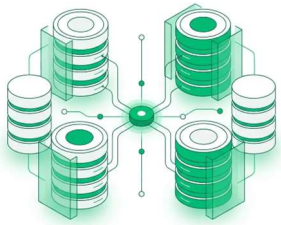
{código_proyecto} referencia PROYECTO

{dni} referencia EMPLEADO

DEPARTAMENTOS (código_dpto, departamento, jefe)



Crear y relacionar tablas



Creación de las Tablas

https://www.w3schools.com/postgresql/postgresql_create_table.php

```
CREATE TABLE nombre_tabla  
(  
    columna1 tipoDatos,  
    columna2 tipoDatos,  
    columna3 tipoDatos,  
    ....  
);
```

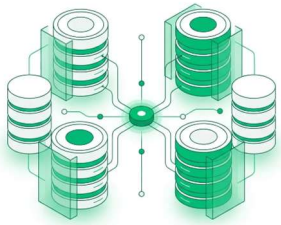
PROYECTOS (id_proyecto, proyecto, descripción)

```
CREATE TABLE proyectos (  
    codigo_proyecto int,  
    proyecto varchar(40),  
    descripcion varchar(255),  
    ....  
);
```

Mostrar la descripción de una tabla:

```
\d nombre_tabla
```

```
\d proyectos;
```

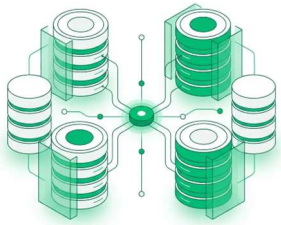



Tipos de Datos PostGres

<https://nubecolectiva.com/blog/tipos-de-datos-en-postgresql-parte-1/>

<https://nubecolectiva.com/blog/tipos-de-datos-en-postgresql-parte-2/>

<https://nubecolectiva.com/blog/tipos-de-datos-en-postgresql-parte-3-final/>



Elementos del Modelo Relacional

Reglas de Integridad

Reglas de **Integridad de Modelo**

Se han de
respetar
SIEMPRE



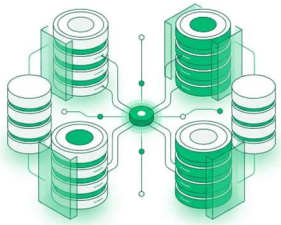
*Restricciones que debe cumplirse
sobre una Base de Datos en todos sus estados.*

Integridad de Entidades:

- Todas las Entidades deben tener una clave primaria
- Regla de **Unicidad** de la Clave Primaria:
La clave primaria que se elija para una entidad, no debe tener valores repetidos.
- Regla de **Entidad** de la Clave Primaria:
Ninguno de los atributos que componen la clave primaria puede ser nulo.

Identidad Referencial:

Para cualquier valor no nulo de una clave foránea deberá existir un valor asociado en la clave primaria de la relación.



Establecer la clave primaria de una tabla

PROYECTOS (código proyecto, proyecto, ...)

PROYECTOS_EMPLEADO (id proyecto+ dni)

```
CREATE TABLE nombre_tabla (  
    columna1 tipoDatos PRIMARY KEY,  
    columna2 tipoDatos,  
    ....  
);
```

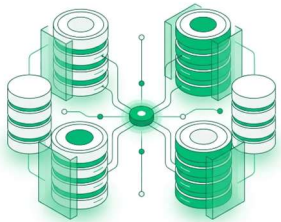
```
CREATE TABLE proyectos (  
    id_proyecto id INT  
        GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    proyecto varchar(40),  
    descripcion varchar(255),  
);
```

```
CREATE TABLE nombre_tabla (  
    columna1 tipoDatos,  
    columna2 tipoDatos,  
    CONSTRAINT PK_tabla  
        PRIMARY KEY (columna(s))  
);
```

```
CREATE TABLE proyectos_empleados (  
    id_proyecto int  
    dni int,  
    CONSTRAINT PK_proyectos_empleados  
        PRIMARY KEY (id_proyecto, dni)  
);
```

```
ALTER TABLE nombre_tabla  
ADD PRIMARY KEY (columna);
```

```
ALTER TABLE proyectos  
ADD PRIMARY KEY (id_Proyecto);
```



Establecer claves candidatas

EMPLEADOS (dni, código, nombre, apellido, fecha_nacimiento, calle/av/via, numero, edf/casa, escalera, planta, puerta, cp, ciudad, sueldo, código_dpto, fecha_inicio, num-sso)

```
CREATE TABLE nombre_tabla (  
    columna1 tipoDatos,  
    columna2 tipoDatos,  
    ....  
    CONSTRAINT nombre_clave  
        UNIQUE (columna)  
);
```

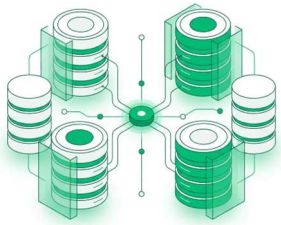
```
ALTER TABLE nombre_tabla  
ADD UNIQUE (columna(s));
```

```
ALTER TABLE nombre_tabla  
ADD CONSTRAINT nombre_clave  
UNIQUE (columna(s));
```

```
CREATE TABLE empleados (  
    dni varchar(10) PRIMARY KEY,  
    codigo int,  
    nombre varchar(20),  
    ....  
    CONSTRAINT codigo_empleado  
        UNIQUE (codigo),  
    CONSTRAINT sso_empleado  
        UNIQUE (num_sso),  
);
```

```
ALTER TABLE empleados  
UNIQUE (num_sso);
```

```
ALTER TABLE empleados  
ADD CONSTRAINT sso_empleado  
UNIQUE (num_sso);
```



Establecer claves foráneas

EMPLEADOS (dni, ..., codigo_dpto)

donde {codigo_dpto} referencia departamento

DEPARTAMENTOS (código dpto, departamento, jefe)

```
CREATE TABLE nombre_tabla (  
    columna1 tipoDatos PRIMARY KEY,  
    columna2 tipoDatos,  
    ....,  
    FOREIGN KEY (columna(s))  
    REFERENCES otra_tabla(columna(s))  
);
```

```
ALTER TABLE nombre_tabla  
ADD FOREIGN KEY (columna(s))  
    REFERENCES otra_tabla(columna(s));
```

```
CREATE TABLE empleados (  
    dni int PRIMARY KEY,  
    ....,  
    FOREIGN KEY (codigo_dpto)  
    REFERENCES departamentos(codigo_dpto)  
);
```

```
ALTER TABLE empleados  
ADD FOREIGN KEY (codigo_dpto)  
    REFERENCES departamentos(codigo_dpto);
```

```
CREATE TABLE [IF NOT EXISTS] nombre_tabla (
  campo_1 tipoDato [opcionesCampo],
  ...
  campo_n tipoDato ★ [opcionesCampo],
  [FOREIGN KEY (campo)
  REFERENCES nombre_tabla(campo)
  [ON DELETE opcionesRelación]
  [ON UPDATE opcionesRelación]]
  [opcionesTabla]
);
```



Clave Primaria (Clave Principal, Identificador)
El campo (o combinación) que permite identificar INEQUÍVOCAMENTE un registro.
Todas las tablas deben tener una clave primaria.

Tabla gatos

Campos o columnas (atributos)

	IdGato	Nombre	Raza	Sexo	Chip
1	1	Kanji	Angora	Macho	111
2	2	Lluna	Persa	Hembra	222
3	3	Taco	Siamés	Macho	333
4	4	Bella	Europeo	Hembra	444

Registros, tuplas o filas

OPCIONES DE CAMPO

NOT NULL Indica si el campo permite valores nulos.
 DEFAULT: Permite establecer un valor inicial por defecto.
 GENERATED ALWAYS AS IDENTITY: Sólo se utiliza en campos de tipo entero.
UNIQUE: Indica que el valor del campo no puede repetirse en toda la tabla.
 PRIMARY KEY: Señala que el campo es Clave Primaria.

OPCIONES DE RELACIÓN

ON DELETE y ON UPDATE permiten indicar el efecto que provocará el borrado o la actualización de los datos en las tabla que referencien a la tabla donde se realiza la operación.
 RESTRICT Impide que se puedan actualizar o eliminar las filas.
 CASCADE Permite actualizar o eliminar las filas.
 SET NULL Asigna el valor NULL a las filas que tienen valores.

delatorrerita es

Tabla visitas

Dato (valor)
La información contenida en cada campo de un registro

	IdGato	Fecha Visita	Motivo
1	1	03/01/2019	Vacunación
1	1	14/01/2019	Baño
2	2	09/01/2019	Vacunación
4	4	17/01/2019	Vacunación
1	1	29/01/2019	Revisión

```
CREATE TYPE sexo_gato
AS ENUM ('Macho', 'Hembra');
```

```
CREATE TABLE gatos (
  idgato INT
  GENERATED ALWAYS AS IDENTITY
  PRIMARY KEY,
  nombre VARCHAR(25) NOT NULL,
  raza VARCHAR(20),
  sexo sexo_gato,
  chip INT UNIQUE
);
```

```
CREATE TABLE visitas (
  idgato INT NOT NULL,
  fecha_visita DATE NOT NULL,
  motivo VARCHAR(100),
  PRIMARY KEY (idgato, fechavisita),
  FOREIGN KEY (idgato)
  REFERENCES gatos (idgato)
  ON UPDATE RESTRICT
  ON DELETE RESTRICT
);
```

Relaciones: Asociaciones entre tablas que se crean para cuidar la integridad de los datos.
 Describen alguna interacción entre dos tablas del modelo.
 Se crean con la combinación CLAVES PRIMARIAS – CLAVES FORÁNEAS

La **Clave Foránea** es un campo (o grupo de campos) que se corresponden con la clave primaria de otra tabla.



	IdGato	Nombre	Raza	Sexo	Chip
	1	Kanji	Angora	Macho	111
	2	Lluna	Persa	Hembra	222
	3	Taco	Siamés	Macho	333
	4	Bella	Europeo	Hembra	444

	idgato	fecha_visita	motivo
	1	03/01/2019	Vacunación
	1	14/01/2019	Baño
	2	09/01/2019	Vacunación
	4	17/01/2019	Vacunación
	1	29/01/2019	Revisión

ALTER TABLE **visitas**

ADD CONSTRAINT visitasgatos

FOREIGN KEY (idgato)

REFERENCES GATOS(idgato)

ON UPDATE

ON DELETE

;

ON UPDATE ... ON DELETE...

RESTRICT ... CASCADE ... SET NULL

Las acciones del ON UPDATE y ON DELETE indican qué sucede con las tablas relacionadas al modificar o borrar registros de la tabla maestra

UPDATE **gatos** SET idgato = 40 WHERE idgato = 4;

DELETE FROM **gatos** WHERE idgato = 2;

ON UPDATE RESTRICT

ERROR 23503: update or delete on table "gatos" violates foreign key constraint "visitasgatos" on table "visitas"

ON UPDATE CASCADE



ON UPDATE SET NULL

idgato	fecha_visita	motivo
1	03/01/2019	Vacunación
1	14/01/2019	Baño
2	09/01/2019	Vacunación
40	17/01/2019	Vacunación
1	29/01/2019	Revisión

idgato	fecha_visita	motivo
1	03/01/2019	Vacunación
1	14/01/2019	Baño
2	09/01/2019	Vacunación
NULL	17/01/2019	Vacunación
1	29/01/2019	Revisión

ON DELETE RESTRICT

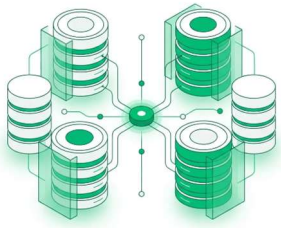
ERROR 23503: update or delete on table "gatos" violates foreign key constraint "visitasgatos" on table "visitas"

ON DELETE CASCADE

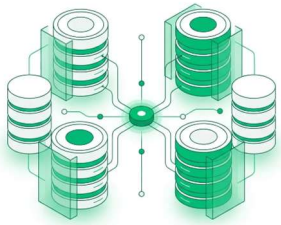
idgato	fecha_visita	motivo
1	03/01/2019	Vacunación
1	14/01/2019	Baño
4	17/01/2019	Vacunación
1	29/01/2019	Revisión

ON DELETE SET NULL

idgato	fecha_visita	motivo
1	03/01/2019	Vacunación
1	14/01/2019	Baño
NULL	09/01/2019	Vacunación
4	17/01/2019	Vacunación
1	29/01/2019	Revisión



Otras restricciones



Elementos del Modelo Relacional

Reglas de Integridad

Restricciones que debe cumplirse sobre una Base de Datos en todos sus estados.

Restricciones Semánticas

Reglas de
Negocio

Son reglas que sirven para definir o restringir alguna acción **en una BBDD en particular.**



Restricción de Unicidad (UNIQUE)

Permite que una clave alternativa o candidata sólo pueda tomar valores únicos (como si de una clave primaria se tratara).

Restricción de Obligatoriedad (NOT NULL),

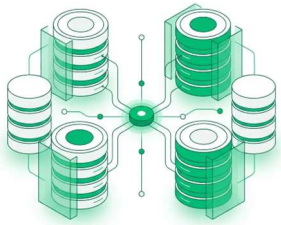
Permite declarar si uno o varios atributos de una tabla, deben tomar siempre un valor.

Restricciones de Dominio (CHECK, campos tipo **Enum**, **UNSIGNED**)

Un dominio se refiere al conjunto de valores que puede tomar un campo.

Restricciones de Usuario

Tema roles y gestión de usuarios



Resumen Lenguaje SQL

<https://modern-sql.com/>

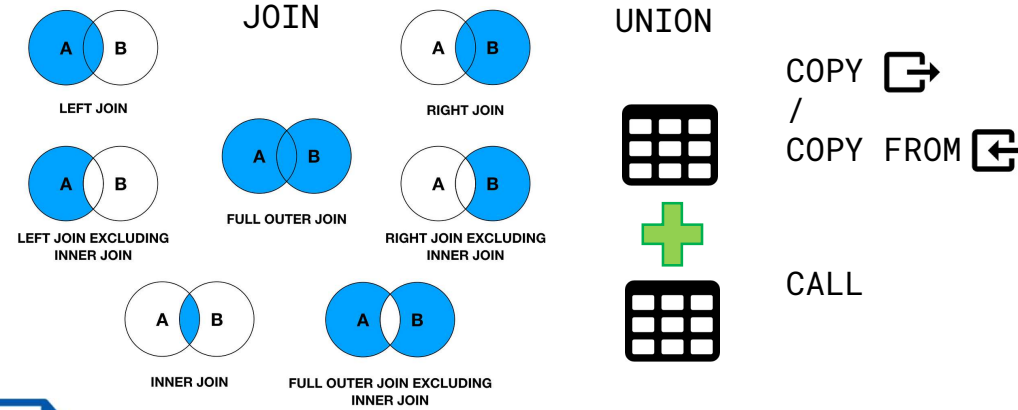
DML: Data Manipulation Language (Tablas)

INSERT
DELETE
UPDATE

SELECT

[ALL | DISTINCT]
[WHERE *condicion(es)*]
[GROUP BY *columna(s)*]
[HAVING *condicion(es)*]
[ORDER BY [ASC | DESC]]

INSERT ... SELECT



★
CONSTRAINT: Relaciones
Tipos de Datos

DDL
Data Definition Language



DCL
Data Control Language

delatorrerita.es

IF EXISTS IF NOT EXISTS	CREATE	ALTER	DROP	TRUNCATE
	Crear	Modificar	Eliminar	Vaciar
DATABASE	X		X	
TABLE ★ [TEMPORARY TABLE]	X	X	X	X
INDEX	X		X	
VIEW	X	X	X	
PROCEDURE	X	X	X	
FUNCTION	X	X	X	
TRIGGER	X		X	

	ROLE	USER
CREATE	X	X
ALTER		X
DROP	X	X
RENAME		X
SET DEFAULT	X	
SET ROLE		X
ALTER ROLE ... WITH PASSWORD		X
GRANT	Otorgar privilegios	
REVOKE	Revocar privilegios	